Documentazione Software

Gestionale Locale per la Gestione del Magazzino

1 Specifica e Requisiti

1.1 Identificazione dei vincoli e analisi del dominio

Questo software ha come obiettivo quello di gestire le dinamiche interne di un azienda che necessiti di un magazzino. Nello specifico abbiamo immaginato un contesto con 3 attori principali: il magazziniere, ovvero colui che si occupa di gestire le merci in entrata e in uscita, l'amministratore, che si occupa di gestire i prezzi e le esportazioni fiscali, e infine il contabile, che si occupa di gestire i documenti contabili. L'applicazione è destinata all'ottimizzazione logistica all'interno di un magazzino merci e deve operare in un contesto puramente locale (offline). Abbiamo valutato tale scelta in modo tale che da rispondere a specifici vincoli ambientali e di sicurezza dell'infrastruttura ospitante, la quale potrebbe presentare zone prive di connettività internet stabile o politiche aziendali restrittive riguardo all'esposizione di dati sensibili all'esterno della rete aziendale.

I vincoli tecnologici imposti prevedono:

- **Ambiente di esecuzione**: Non ci sono requisiti di compatibilità specifici. Il software deve poter essere fruibile da un qualsiasi sistema operativo desktop moderno
- **Archiviazione dati**: Utilizzo di un sistema di gestione di basi di dati incorporato (embedded), identificato in MySQL (???).
- **Interfaccia**: Sviluppo di un'interfaccia grafica leggera e reattiva che non richieda risorse hardware elevate per l'esecuzione (???).

1.2 Requisiti Funzionali

I requisiti funzionali descrivono in modo dettagliato le azioni che il sistema deve consentire e i servizi che deve offrire all'utente finale. Nel contesto del gestionale di magazzino, si identificano le seguenti macro-funzionalità:

- Il login dei vari attori con permessi diversi. Abbiamo valutato di inserire una forma di sicurezza basata su passfrase in caso l'utente si sia dimenticato della propria password
- Il magazziniere deve poter vedere sia le merci in entrata che quelle in uscita
- Il magazziniere deve poter inserire le merci in entrata e in uscita, modificare o anche solo visionare le giacenze

- Il magazziniere deve poter produrre facilmente un DDT (documento di trasporto) per ogni merce in uscita. Allo stesso tempo l'ordine passa dallo stato "Non evaso" a "Evaso" (lo reistriamo???)
- Il magazziniere deve poter controllare un reso (sia come elenco sia nel dettaglio)
- Il contabile deve poter visionare sia come elenco che nel dettaglio gli ordini ricevuti, deve poterli filtrare per evasi e non evasi e deve poter generare una fattura
- Il contabile deve poter essere in grado di modificare i prezzi di vendita e di acquisto
- Il contabile deve poter visualizzare e gestire ogni reso. Se necessario, deve poter generare una nota di credito
- L'amministratore deve poter visionare le merci in magazzino e i relativi prezzi, deve poter produrre e/o controllare le fatture e deve poter gestire i resi

1.3 Requisiti Non Funzionali

I requisiti non funzionali definiscono le caratteristiche qualitative del sistema e i limiti entro i quali deve operare per garantire standard elevati di affidabilità, sicurezza e usabilità. Di seguito proponiamo un elenco di requisiti non funzionali specifici e relativi ai funzionali sopra descritti:

- Il sistema riceve un ordine sottoforma di elenco con: Id ordine, ordinante, Id oggetto, quantità oggetto richiesto, data ordine, ordine evaso o non evaso
- Il sistema deve strutturare la giacenza degli oggetti presenti nel magazzino. Deve poterli catalogare in funzione delle categorie: Id oggetto, nome oggetto, quantità, prezzo di vendita, gruppi di filtraggio, attivo o non attivo. Il sistema deve inoltre poter eliminare un oggetto
- Il sistema deve poter generare un DDT con le informazioni del cliente relative a: dati cliente, trasportatore, merci e quantità. Inoltre il sistema cambia lo stato dell'ordine da "Non evaso" a "Evaso"
- Il sistema deve modificare le quantità di un oggetto presente in magazzino di cui sia stato fatto un reso
- Il sistema deve generare un documento di vendita con i dati del cliente e le quantità degli ordini. La fattura deve essere registrata con progressivo (numero???) e anno e deve essere conservata
- Il sistema deve calcolare automaticamente il prezzo di vendita di un oggetto in funzione del prezzo di acquisto x la quantità richiesta. Il sistema deve poter dare il valore totale corrente del magazzino
- Il sistema deve poter fornire i dati della fattura in maniera automatica estrapolandoli da database. Deve, inoltre, permettere la modifica manuale
- Il sistema deve poter mostrare i dati su un reso effettuato e deve mostrare se è stato elaborato o meno

- Il sistema deve poter stampare i report per merci nel magazzino e relativo valore, le fatture emesse e i resi effettuati ad uso esclusivo dell'utente con permessi di amministratore

?????

1.4 Requisiti Misurabili (Metriche)

Gli obiettivi qualitativi espressi in modo vago devono essere convertiti in metriche oggettive, quantificabili e verificabili in fase di collaudo:

- **Tempo di risposta alla scansione**: A seguito della lettura di un codice a barre tramite lettore ottico, il sistema deve completare la decodifica, effettuare l'aggiornamento della giacenza nel database locale e aggiornare l'interfaccia grafica in un tempo inferiore a 0.5 secondi per il 95% delle operazioni, ipotizzando una base dati contenente fino a 50.000 articoli distinti.
- **Efficienza della reportistica**: La generazione di un report annuale completo dell'inventario, comprensivo di valorizzazione economica totale del magazzino, non deve richiedere più di 3 secondi di elaborazione CPU.
- **Impronta di memorizzazione**: L'eseguibile dell'applicazione e le sei dipendenze non devono occupare più di 200 MB di spazio su disco rigido all'atto dell'installazione. L'incremento del file di database deve mantenersi lineare, stimato in meno di 10 MB ogni 10.000 transazioni registrate storicamente.

1.5 Livelli di Requisiti

La formulazione dei requisiti viene strutturata secondo due distinte prospettive per garantire l'allineamento tra le necessità del business e l'implementazione tecnica:

- **Lato utente**: L'operatore del magazzino deve ricevere una notifica visiva immediata e persistente (es. un banner rosso o un'icona di avviso accanto all'articolo) non appena la disponibilità fisica di un determinato prodotto scende al di sotto della sua soglia minima di sicurezza preimpostata, al fine di prevenire situazioni di stock-out.
- **Lato sistema**: La logica di back-end deve agganciarsi al modulo delle transazioni del database. Ad ogni operazione di *Scarico*, il sistema esegue una query di verifica sulla tabella `Articoli`; se il valore del campo `quantita_attuale` risulta inferiore al valore del campo `soglia_minima`, il sistema deve impostare un flag booleano interno (`stato_sotto_soglia = true`) e scatenare un evento software asincrono. Tale evento verrà intercettato dal componente della View per attivare dinamicamente l'allarme visivo sull'interfaccia utente.

2 Artefatti di Pianificazione Agili (Scrum)

???

2.1 Backlog

L'elenco ordinato dei requisiti tecnici e funzionali necessari alla realizzazione del sistema unito alle user stories generate in fase di sviluppo comprende:

1. Configurazione dell'architettura software di base, dell'ambiente di sviluppo e della connessione al database MySQL.
2. Definizione dello schema logico della base dati (creazione delle tabelle `Articoli`, `Movimenti`, `Utenti`, `Fornitori`).
3. Sviluppo delle interfacce grafiche e delle funzioni CRUD per la gestione dell'Anagrafica Articoli.
4. Implementazione del modulo di movimentazione (logica e maschere per le operazioni di Carico e Scarico).
5. Integrazione software per l'acquisizione dell'input da lettore di codici a barre tramite emulazione tastiera.
6. Sviluppo del sottosistema di controllo e notifica visiva per gli articoli sotto la soglia minima di scorta.
7. Progettazione ed implementazione del motore di reportistica ed esportazione dati in formato PDF e CSV.
8. Sviluppo del modulo di autenticazione locale e gestione dei permessi basata sui ruoli (RBAC).
9. Implementazione delle routine software per l'esecuzione del backup automatico e del ripristino del database.

2.2 User Stories

Le storie riflettono i bisogni concreti degli attori del sistema scritti secondo la formula standard del framework Scrum:

> «Come *magazziniere*, voglio *poter scansionare il codice a barre di un articolo in arrivo tramite il lettore ottico*, per *aggiornare la quantità in giacenza nel sistema in modo rapido, automatizzato e senza il rischio di commettere errori manuali di digitazione*».

?????

2.3 Criteri di Accettazione

I criteri di accettazione definiscono i vincoli formali affinché la User Story sopra citata possa essere considerata completata ("Done"):

- **Criterio 1**: Il sistema deve riconoscere e decodificare correttamente e senza latenza i codici a barre conformi allo standard internazionale EAN-13.

- **Criterio 2**: Al momento della scansione riuscita all'interno della schermata di carico, i dati identificativi dell'articolo devono comparire istantaneamente nei campi visivi e la relativa giacenza nel database deve essere incrementata automaticamente di una unità (o del valore impostato nel moltiplicatore di input).
- **Criterio 3**: Qualora il codice a barre scansionato non trovi corrispondenza all'interno della tabella `Articoli`, il sistema deve inibire il salvataggio automatico, emettere un feedback acustico di errore ed aprire una finestra pop-up contestuale che consenta all'operatore di inserire l'articolo ex-novo in anagrafica o annullare l'operazione in corso.

3 Documentazione Tecnica e di Progetto

Questo capitolo delinea le scelte architetturali interne del software e i criteri di progettazione volti a prevenire il debito tecnico e garantire la manutenibilità a lungo termine.

3.1 Decisioni Progettuali

La scelta strategica di utilizzare SQLite come motore di database principale risponde direttamente al vincolo di esecuzione locale e portabilità. A differenza di architetture client-server, SQLite compila l'intera base dati in un singolo file binario residente nella cartella dell'applicazione. Questa decisione riduce a zero i prerequisiti di installazione sulle macchine client, non richiede l'attivazione di demoni o servizi di rete che consumano risorse in background, e semplifica drasticamente le procedure di manutenzione: l'intero sistema può essere spostato o salvato tramite una semplice operazione di copia del file di database.

3.2 Documentazione Mirata (Back-end)

Il codice sorgente dell'applicazione adotta rigorosamente il pattern architetturale **Model-View-Controller (MVC)**, strutturando le dipendenze in modo unidirezionale per evitare accoppiamenti forti:

- **Model (Modello)**: Comprende le classi rappresentanti le entità di dominio (`Articolo`, `Movimento`, `Utente`) e lo strato dei Data Access Object (DAO). Le classi DAO incapsulano la totalità delle istruzioni SQL, gestendo l'apertura e la chiusura delle connessioni al file SQLite e garantendo l'isolamento della persistenza.
- **Controller (Controllore)**: Costituisce il nucleo della logica di business (business logic). Si occupa di validare i dati di input, coordinare i flussi operativi, calcolare le valorizzazioni inventariali e invocare i DAO appropriati. Il Controller non conosce i dettagli dell'interfaccia grafica.
- **View (Vista)**: Costituita dai componenti dell'interfaccia grafica utente (GUI). Ha il solo compito di renderizzare i dati inviati dal Controller e catturare gli input fisici (tastiera, mouse, scanner) per inoltrarli al rispettivo Controllore, senza eseguire alcuna query diretta o calcolo computazionale sui dati.

3.3 Progettazione dei Test

Al fine di garantire la robustezza logica del software rispetto a input errati o anomalie operative, viene definita una suite di test di unità (Unit Testing). Il test cardine per la sicurezza del magazzino è focalizzato sulla prevenzione delle giacenze negative:

> *Scenario di test*: Supponendo che un articolo registrato in anagrafica disponga di una quantità fisica in giacenza pari a 5 unità, se viene invocata la funzione di business `registraScarico()` richiedendo l'uscita di 6 unità, il sistema deve intercettare preventivamente l'incoerenza, bloccare l'esecuzione della transazione sul database tramite un'operazione di `ROLLBACK` e sollevare un'eccezione software personalizzata di tipo `GiacenzaInsufficienteException`. L'interfaccia utente dovrà catturare tale eccezione per visualizzare un messaggio di errore bloccante, impedendo che i dati di magazzino assumano valori negativi incongruenti.

4 Documentazione di Processo (es. Modello Waterfall)

Nei contesti di sviluppo tradizionali o regolamentati, la produzione documentale iniziale costituisce il cardine per la tracciabilità delle scelte progettuali e la stipula del contratto operativo con gli stakeholder.

4.1 Documentazione iniziale (Forte)

La documentazione redatta prima della fase di codifica assume un valore vincolante e strutturato per allineare i team di sviluppo e le figure non tecniche della committenza. Essa è costituita da:

1. **Diagramma Entità-Relazione (ER)**: Lo schema concettuale e logico dettagliato che definisce le tabelle del database, descrivendo i primi tipi di dato di ciascun attributo, le chiavi primarie, le chiavi esterne e i vincoli di integrità referenziale (es. la cancellazione a cascata dei movimenti al variare di un articolo, se ammessa, o il blocco restrittivo).

2. **Wireframe e Storyboard delle Interfacce**: Documentazione visiva statica che mappa l'esatta disposizione dei componenti grafici (campi di testo, pulsanti, griglie dati) di ogni schermata dell'applicativo, descrivendo le transizioni logiche da una finestra all'altra a seguito di eventi utente.

3. **Specifica dei Requisiti di Sistema**: Elenco dei requisiti hardware e software minimi richiesti sulla macchina ospitante per garantire l'esecuzione corretta dell'applicazione (es. architettura CPU x86/x64 da almeno 1.6 GHz, 4 GB di RAM libera, 500 MB di spazio disponibile su disco a stato solido, e la versione specifica del framework di runtime o della Java Virtual Machine/.NET CLR).

Questo patrimonio informativo garantisce la possibilità di condurre audit formali sulla qualità del software, facilita l'onboarding di nuovi sviluppatori nel team e protegge il progetto da derive dei requisiti (scope creep).